

281

Program testing

- The purpose of testing is to show that a program does what it is intended to do and to discover program defects before it is put into use.
- When you test software, you execute a program using artificial data.
- You check the results of the test run for errors, anomalies or information about the program's non-functional attributes.
- Tests can reveal the presence of errors NOT their absence.
- Testing is part of a more general verification and validation (V&V) process.

282

Reviews and testing

- **Software reviews** Concerned with analysis of the static system representation to discover problems (static verification)
  - May be supplemented by tool-based document and code analysis.
- **Software testing** Concerned with exercising and observing product behaviour (dynamic verification)
  - The system is executed with test data and its operational behaviour is observed.

Reviews and testing

- Reviews and testing are complementary and not opposing verification techniques.
- Both should be used during the V&V (Verification and Validation) process.
- Reviews can check conformance with a specification but not conformance with the customer's real requirements.
- Reviews cannot check non-functional characteristics such as performance, usability, etc.

Unit testing

- Unit testing is the process of testing individual components in isolation.
- It is a defect testing process.
- Units may be:
  - Individual functions or methods within an object
  - Object classes with several attributes and methods
  - Composite components with defined interfaces used to access their functionality.

Testes unitários ?

- são uma forma de verificar se pequenas partes (unidades) do código estão a funcionar correctamente
- As unidades podem ser métodos, classes ou pequenas funcionalidades isoladas.
- A ideia é testar cada componente de forma independente para garantir que está a produzir os resultados esperados

A importância dos testes unitários

- Detecção precoce de erros/"bugs":
  - Ao escrever testes unitários à medida que desenvolve o código, consegue-se identificar e corrigir os erros de uma forma mais rápida, evitando que se propaguem para outras partes do sistema.
- Melhora da qualidade do código:
  - Testes unitários são um incentivo a escrever código mais "limpo", modular e fácil de testar, o que, por sua vez, torna o código mais fiável e de fácil manutenção.
- Aumento da confiança:
  - Com uma boa cobertura de testes, há mais confiança ao fazer se alterações no código, uma vez que os testes garantem que as novas funcionalidades não prejudicam as existentes.
- Facilita a optimização do software
  - Quando se procede a uma optimização do código os testes permitem saber se essa optimização tem impacto negativo no funcionamento do mesmo.

287

Boas práticas na implementação de testes unitários (1)

- Testes independente
  - Cada teste deve ser independente dos outros. Ou seja o resultado de um teste não deve influenciar o resultado de outro teste
- Testar casos limites
  - Além dos casos comuns, é importante testar os limites, por exemplo, como valores nulos, negativos, extremos .... E assim garantir-se que a unidade resolve bem estas situações

288

Boas práticas na implementação de testes unitários (2)

- Abrangência
  - O grau de abrangência mede quanto código foi submetido a testes unitários
  - O importante é garantir que as funcionalidades críticas estejam testadas de forma conveniente, mais do que ter 100% !!
- Execução rápida
  - Como os testes unitários são executados várias vezes ao longo do processo de desenvolvimento o tempo de execução deve ser minimizado.
  - Testes lentos influenciam a produtividade e provoca o não uso …

289

289

Exemplos de Ferramentas para Testes Unitários

- Existem várias ferramentas para escrever e executar testes unitários em diferentes linguagens de programação:
  - Python:
    - unittest, pytest
  - **Java:**
    - **Junit**
  - JavaScript/TypeScript:
    - Jest, Mocha
  - C#:
    - NUnit, xUnit
- Estas ferramentas permitem a definição de testes de forma estruturada e fornecem relatórios sobre o resultado (o que foi testado, o que passou e o que falhou).

290

290

Testes unitários com o GitLab CI/CD

- A integração de **testes unitários no pipeline CI/CD (Continuous Integration/Continuous Delivery**) do GitLab é uma prática comum para garantir a qualidade do código a cada alteração.
- Com GitLab CI/CD, os testes unitários podem ser executados automaticamente sempre que (por exemplo):
  - Alguém faz um **push** de código para o repositório.
  - um **merge request** para integrar código numa branch principal (ex.: main ou develop).

291

291

Benefícios da Automação dos Testes com GitLab

- Há garantias que os testes são executados de forma consistente, sem depender da execução manual
- Facilita a identificação de falhas antes de integrar as novas funcionalidades ou as correcções de erros, mantendo a estabilidade do código.
- Feedback Rápido
  - feedback quase imediato sobre a qualidade do código após cada alteração, permitindo, assim, ajuste/correcções imediatas.

292

292

Integration testing

- In integration testing individual software modules are combined and tested as a group.
- It occurs after unit testing and before system or acceptance testing.

1. **In incremental integration testing,** the developers integrate the modules one by one

2. **Big bang integration** is another integration testing technique, where all the modules are integrated in one shot (of course, you should avoid this!)

Objectivo dos Testes de Integração

- identificar problemas que possam surgir quando diferentes partes do sistema interagem entre si.
- Muitas vezes, uma função ou módulo pode funcionar correctamente de forma isolada, mas falhar ao interagir com outros devido a problemas como incompatibilidade de dados, diferenças de formato, ou erros de comunicação.

Quando e Como Realizar Testes de Integração

- Os testes de integração são geralmente realizados após os testes unitários e antes dos testes de aceitação ou de sistema.
- Normalmente, estes testes seguem o processo de integração do sistema, em que componentes de código, bibliotecas externas, APIs e bases de dados são combinados em grupos maiores para verificar se os subsistemas comunicam correctamente.

295

295

Abordagens para os testes de integração

- **Integração Incremental**
  - Testa os módulos ou componentes progressivamente, adicionando um módulo de cada vez e verificando a integração com os que já foram testados.
    - Bottom-up
      - Começa pelos módulos de nível mais baixo e vai testando em direção aos módulos de nível superior.
    - Top-down:
      - Começa com os módulos de nível superior e vai descendo.
- **Integração Big Bang**
  - Todos os módulos são integrados e testados ao mesmo tempo. Esta abordagem permite testar o sistema completo de imediato, mas pode dificultar a identificação de problemas específicos entre módulos.

296

296

Ferramentas de Teste de Integração

- Existem várias ferramentas que facilitam a execução de testes de integração
  - JUnit (para Java), pytest (para Python)
    - Usadas para escrever testes de integração que envolvem vários módulos.
  - Postman ou SoapUI
    - Para testar integrações entre serviços e APIs.
- Sistemas de Integração Contínua (CI) como GitLab CI/CD
  - Automatizam a execução de testes de integração a cada alteração no código.

297

297

Importância dos Testes de Integração

- Os testes de integração são fundamentais para garantir que o sistema funciona correctamente como um todo e que os componentes individuais colaboram de forma eficiente.

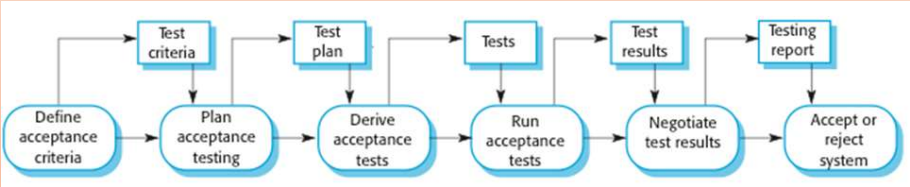
- Identificar e corrigir problemas de comunicação entre módulos.
- Garantir que as dependências externas, como APIs ou bases de dados, são integradas correctamente.
- **Reduzir o risco de falhas em produção, oferecendo uma maior confiança na funcionalidade do sistema.**

298

298

Acceptance testing

- Acceptance testing is a user testing process where the aim is to decide if the software is good enough to be deployed and used in its operational environment.



299

Objectivo dos Testes de Aceitação

- validar que o sistema está **pronto para ser utilizado** pelo utilizador final. Ou seja, os testes verificam se o software cumpre todos os critérios definidos pelo cliente ou utilizador, incluindo requisitos funcionais e não funcionais, como desempenho, usabilidade e compatibilidade.

- Enquanto os testes de integração verificam a comunicação entre componentes, os testes de aceitação focam-se em **simular o uso real da aplicação** para garantir que todas as funcionalidades se comportam de acordo com os requisitos estabelecidos.

300

Tipos de Testes de Aceitação (1)

- Existem vários tipos de testes de aceitação que podem ser aplicados, dependendo dos objectivos do projecto

1. Teste de Aceitação do Utilizador (UAT - User Acceptance Testing)
   - Realizado pelos utilizadores finais ou clientes, normalmente em ambientes que simulam as condições reais de produção.
   - Verifica se o sistema é funcional e se os utilizadores conseguem completar as suas tarefas conforme esperado.
2. Teste de Aceitação Operacional
   - Avalia aspectos operacionais do sistema, como desempenho, segurança e estabilidade, garantindo que o sistema é confiável e está pronto para a produção.
   - Verifica se o sistema se comporta bem em cenários de carga e stress, e se os backups, logging e outros sistemas operacionais estão em pleno funcionamento.

301

301

Tipos de Testes de Aceitação (2)

3. Teste de Aceitação Contratual
   - Valida que todos os requisitos e critérios especificados no contracto ou no acordo com o cliente foram cumpridos.
   - Normalmente, esta validação é feita por equipas de qualidade ou auditores, garantindo que o software entregue cumpre exactamente o que foi acordado.
4. Teste Beta
   - Envolve o lançamento do software para um grupo seleccionado de utilizadores finais que vão testar a aplicação em cenários reais.
   - Os utilizadores fornecem feedback sobre a funcionalidade e usabilidade, ajudando a identificar problemas que podem não ter surgido durante o desenvolvimento.

302

302

Importância dos Testes de Aceitação

- Os testes de aceitação são críticos para assegurar que o software cumpre o objectivo final e satisfaz as necessidades do cliente.
- Principais vantagens, temos:
  - **Validação pela Perspetiva do Utilizador**: Os testes de aceitação colocam o foco na experiência do utilizador, garantindo que o software cumpre os requisitos e é intuitivo e funcional.
  - **Confirmação do Cumprimento dos Requisitos**: Estes testes certificam que todos os critérios definidos no início do projecto foram atendidos, evitando surpresas após o lançamento.
  - **Redução de Riscos**: Identificar problemas antes do lançamento evita custos elevados com correcções em produção e aumenta a satisfação do cliente final.

303

Agile – Acceptance Criteria

- Conditions that must be satisfied for a product, user story, or increment of work to be accepted.
- Expressed as a set of statements:
  - Clear, so that everyone understands them
  - Concise, so that there's no ambiguity
  - Testable or verifiable
  - Focused on providing customer-delighting results
- Acceptance criteria complement User stories

304

Usability testing

- Usability testing refers to evaluating a product or service by testing it with representative users.
- Typically, during a test, participants will try to complete typical tasks while observers watch, listen and take notes.
- The goal is to identify any usability problems, collect qualitative and quantitative data and determine the participant's satisfaction with the product.

testes de usabilidade

- tipo de testes focados em avaliar a **experiência do utilizador** ao interagir com o sistema ou aplicação
- Diferente dos testes de aceitação ou testes funcionais, que verificam se o software cumpre os requisitos e funciona corretamente, os testes de usabilidade avaliam a **facilidade de uso**, a **intuitividade** e a **satisfação do utilizador**.
- *Os testes de usabilidade são especialmente importantes em aplicações voltadas para o consumidor final, onde uma má experiência de utilizador pode levar à insatisfação e ao abandono do produto*

Importância dos Testes de Usabilidade

Os testes de usabilidade são essenciais para:

- **Melhorar a experiência do utilizador**
  - Identificar e corrigir problemas que poderiam frustrar ou confundir os utilizadores.
- **Aumentar a taxa de retenção**
  - Um sistema mais fácil de usar e intuitivo incentiva os utilizadores a continuarem a utilizá-lo.
- **Reduzir o suporte ao cliente**
  - Interfaces intuitivas e autoexplicativas levam a menos dúvidas e menos chamadas de suporte.

307

Usability test plan

- Should include:
  - **Scope** - Indicate what you are testing. E.g. the navigation and content.
  - **Purpose** - Identify the concerns, questions, and goals for this test. Example: "Can users navigate to important information from the prototype's home page?"
  - **Participants** - Number and types of participants
  - **Scenarios** - describe the stories and context behind why a specific user or user group comes to your application. They note the goals and questions to be achieved and sometimes define the possibilities of how the user(s) can achieve them on the application. Example: You are traveling to Bragança for your job next week and you want to check on the amount you can be reimbursed for meals and other expenses
  - **Metrics** (e.g., successful completion rates, error rates, time on task)

308

Further reading

- Andrew Stellman and Jennifer Greene, *Applied Software Project Management*, O'Reilly, 2006. [Section 8.1]
- *Usability testing*, online: https://www.usability.gov/how-to-and-tools/methods/usability-testing.html
- Some slides adapted from A. Stellman & J.Greene, "Applied Software Project Management" and Ian Sommerville, "Software Engineering"
- Everything You Need to Know About Acceptance Criteria, BY SCRUM ALLIANCE, https://resources.scrumalliance.org/Article/need-know-acceptance-criteria

309

Gestão de Projecto de Software

Licenciatura em Engenharia Informática

2024/2025

310